

One-Page Report: Python Web App with Docker Compose

Flask + Redis containerized application | Implementation and verification

1. Objective

The objective was to build and run a simple Python web application using Docker Compose. The application uses Flask for the web service and Redis as a separate service to persist and increment a page-visit counter. Docker Compose manages both services as a single application stack.

2. Implementation

Component	Configuration / Role
requirements.txt	Flask 3.0.3 and Redis Python client 5.0.7
app.py	Flask route at /; Redis counter with connection-retry logic
Dockerfile	Python 3.12 Alpine image; installs dependencies; exposes container port 5000
compose.yaml	web service builds the image and maps 8000:5000; Redis uses redis:alpine

3. Build and Run

The stack was started from the VS Code terminal with **docker compose up**. The screenshots show the web image installing Flask/Redis dependencies, Redis starting successfully, and the Flask application serving on 0.0.0.0:5000 inside the container. The Compose port mapping makes the application available from the host at **http://localhost:8000**.

2024-09-01 22:11:02 postgres The files belonging to this database ...	2024-09-01 22:36:18 webapp Collecting click=8.1.3 (from Flask)	2024-09-01 22:36:23 webapp/redis Starting Redis Server	2024-09-01 22:36:23 webapp/web * Serving Flask app 'app.py'
2024-09-01 22:11:02 postgres This user must also own the server	2024-09-01 22:36:18 webapp Downloading click-8.1.6-py3-none-ar	2024-09-01 22:36:23 webapp/redis 1:C 02 Sep 2024 02:36:23.155 * OKC	2024-09-01 22:36:23 webapp/web * Debug mode: off
2024-09-01 22:11:02 postgres The database cluster will be initi	2024-09-01 22:36:18 webapp Collecting blinker=1.6.2 (from fl	2024-09-01 22:36:23 webapp/redis 1:C 02 Sep 2024 02:36:23.155 * Repl	2024-09-01 22:36:23 webapp/web WARNING: This is a development serv
2024-09-01 22:11:02 postgres The default database encoding has e	2024-09-01 22:36:18 webapp Downloading blinker-1.6.2-py3-none-	2024-09-01 22:36:23 webapp/redis 1:C 02 Sep 2024 02:36:23.155 * Conf	2024-09-01 22:36:23 webapp/web * Running on all addresses (0.0.0.0)
2024-09-01 22:11:02 postgres The default test search configurat	2024-09-01 22:36:18 webapp Collecting MarkupSafe=2.0 (from J	2024-09-01 22:36:23 webapp/redis 1:M 02 Sep 2024 02:36:23.155 * Mon	2024-09-01 22:36:23 webapp/web * Running on http://127.0.0.1:5000
2024-09-01 22:11:02 postgres data page checksums are disabled.	2024-09-01 22:36:18 webapp Downloading MarkupSafe-2.0-py312-	2024-09-01 22:36:23 webapp/redis 1:M 02 Sep 2024 02:36:23.152 * Run	2024-09-01 22:36:23 webapp/web * Running on http://127.0.0.1:5000
2024-09-01 22:11:02 postgres fixing permissions on existing dir	2024-09-01 22:36:18 webapp Downloading flask-3.0.3-py3-none-ar	2024-09-01 22:36:23 webapp/redis 1:M 02 Sep 2024 02:36:23.152 * oBf	2024-09-01 22:36:23 webapp/web Press CTRL-C to quit
2024-09-01 22:11:02 postgres creating subdirectories ... ok	2024-09-01 22:36:18 webapp Downloading redis-5.0.7-py3-none-ar	2024-09-01 22:36:23 webapp/redis 1:M 02 Sep 2024 02:36:23.152 * oBf	2024-09-01 22:37:11 webapp/web 192.168.45.1 - - [02/Sep/2024 02:37
2024-09-01 22:11:02 postgres selecting dynamic shared memory tpe	2024-09-01 22:36:18 webapp Downloading redis-5.0.7-py3-none-	2024-09-01 22:36:23 webapp/redis 1:M 02 Sep 2024 02:36:23.152 * oBf	2024-09-01 22:37:11 webapp/web 192.168.45.1 - - [02/Sep/2024 02:37
2024-09-01 22:11:02 postgres selecting default max_connections	2024-09-01 22:36:18 webapp Downloading blinker-1.6.2-py3-none-	2024-09-01 22:36:23 webapp/redis 1:M 02 Sep 2024 02:36:23.152 * oBf	2024-09-01 22:37:41 webapp/web 192.168.45.1 - - [02/Sep/2024 02:37
2024-09-01 22:11:02 postgres selecting default shared_buffers	2024-09-01 22:36:18 webapp Downloading MarkupSafe-2.0-py312-	2024-09-01 22:36:23 webapp/redis 1:M 02 Sep 2024 02:36:23.152 * oBf	
2024-09-01 22:11:02 postgres selecting default time zone ... Etc	2024-09-01 22:36:18 webapp Installing collected packages: red		

Evidence: dependency installation and image build (left), Redis service startup (middle), and Flask web service running with requests reaching the container (right).

4. Result and Verification

The deployment completed successfully with two running services: **webapp/web** and **webapp/redis**. Redis starts as the backing service, while Flask serves the application. The web-container logs show successful HTTP requests from the host, confirming that the application was reachable through the published port. Refreshing the browser increments the visit counter because the value is stored in Redis.

5. Conclusion

Docker Compose successfully packaged the Flask application and Redis dependency into a reproducible multi-container environment. The separation of the web and data services demonstrates a basic microservice-style architecture and makes the application easy to start or stop as one stack. The stack can be stopped with **Ctrl+C** and cleaned up with **docker compose down**.